

DM à rendre, mais essentiellement auto-corrigé :

- Pour les questions de programmation, un fichier de test, essentiellement identique à celui que j'utiliserai pour corriger, se trouve sur la page du cours. Afin de vérifier que tout marche bien, vous placerez les fichiers `tri_rapide.ml` (modifié par vos soins), `tri_rapide.mli` et `test.ml` dans le même dossier, puis vous compilerez et exécuterez :

```
ocamlopt tri_rapide.mli
ocamlopt -I +unix unix.cmx tri_rapide.ml test.ml
./a.out
```

Vous pouvez ajouter des arguments au programme, comme indiqué dans `test.ml`

Le fichier `tri_rapide.ml` est à rendre pour le Vendredi 9 janvier 23h59.

- Le Vendredi 9 janvier, je ramasserai la partie algorithmique du DM.
- Je corrigerai cinq DMs choisis au hasard et la note comptera comme une note de colle.

Le but de ce DM est d'écrire une version déterministe de l'algorithme de tri rapide, tout en gardant une complexité dans le pire cas $\Theta(n \log(n))$.

Les questions peuvent être traitées indépendamment quitte à admettre le résultat des questions précédentes, sauf la question 2 qui utilise la question c), les questions 8 et 10 qui dépendent de la question 7, et les questions de la partie 4 qui sont nécessaire à la dernière question de cette partie.

1 Préliminaires

On se basera dans ce problème sur la version du tri rapide disponible dans le dossier du DM sur le site de la classe. Il s'agit de la version de la question 4 de l'exercice 4 du TP 13, basée sur l'algorithme du drapeau néerlandais pour la partie "Diviser". À chaque appel de la fonction, le pivot est choisi uniformément au hasard parmi les éléments à trier.

- On pose $\mathbf{a} = [3; 4; 42; 6; 57; 13; 0; 9; 0; -1; 8]$, $\mathbf{deb} = 0$, $\mathbf{fin} = 6$, $\mathbf{pivot} = 6$.
 - On considère l'appel `drapeau_neerlandais a pivot deb deb (fin - 1)` comme à la ligne 47 du programme. Donner tous les appels récursifs faits lors de l'exécution de cette fonction, en précisant à chaque fois l'état du tableau $\mathbf{a}$ (il n'est pas nécessaire de réécrire à chaque fois ce qui ne change pas).
 - Vérifier dans l'exemple précédent que la propriété illustrée aux lignes 15 à 19 du programme est bien vérifiée à chaque appel récursif de `drapeau_neerlandais`.
 - Formuler mathématiquement cette propriété, et justifier qu'elle est toujours vérifiée par l'appel de la fonction à la ligne 47.
- Prouver, pour tous indices $j \leq k$ du tableau $\mathbf{a}$, par récurrence sur $m = k - j$, que si ses arguments vérifient la propriété énoncée à la question c), la fonction `drapeau_neerlandais` termine et est correcte.
- Prouver que la complexité de cette fonction dans le pire cas est $\Theta(m)$, toujours avec $m = k - j$.

On a vu (ou on verra) en cours que :

- si (par malheur) $\mathbf{pivot}$ est toujours le minimum des éléments à trier, alors la complexité de l'algorithme sur un tableau de taille n est $\Theta(n^2)$;
- la complexité moyenne de l'algorithme de tri rapide, si on se place dans le modèle où tous les éléments du tableaux sont distincts, est $\Theta(n \log(n))$.

2 L'oracle de la médiane

On suppose dans cette section qu'on remplace la ligne 46 du programme par

```
1 let pivot = mediane a deb fin in
```

où `mediane a deb fin` est une médiane¹ des éléments de $\mathbf{a}$ d'indice $i \in [\mathbf{deb}; \mathbf{fin} - 1]$.

De plus, on suppose que la fonction `mediane` a une complexité dans le pire cas linéaire en la différence $\tilde{n} = \mathbf{fin} - \mathbf{deb}$.

- Montrer que avec cette substitution, la complexité dans le pire cas du tri rapide vérifie

$$C(n) = C\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + C\left(\left\lceil \frac{n}{2} \right\rceil - 1\right) + \mathcal{O}(n).$$
- En déduire que la complexité dans le pire cas (toujours avec l'hypothèse qu'on dispose d'un oracle linéaire pour la médiane) du tri rapide est $\mathcal{O}(n \log(n))$.

1. Pour clarifier : il s'agit nécessairement d'un élément du tableau. S'il y a un nombre pair d'éléments, on prend un des deux éléments candidats, pas leur moyenne arithmétique. La médiane de $[1; 3; 5; 6]$ sera donc 3 ou 5, mais pas 4.

3 Médiane des médianes

Trouver une médiane d'un tableau en temps linéaire n'est pas du tout trivial. En effet il est facile de *vérifier* qu'une valeur est une médiane par un simple parcours en comptant le nombre d'éléments strictement supérieurs et strictement inférieurs. Mais la trouver est plus ardu si on ne s'autorise pas à trier le tableau auparavant (ce qui est ce qu'on cherche au final à faire, nous aurions un problème de poule et d'œuf).

On utilise un algorithme appelé "médiane des médianes", nom légèrement trompeur comme on va le voir ci-dessous. C'est un algorithme diviser pour régner un peu particulier.

L'algorithme prend en argument un tableau A (ou, en pratique, une portion d'un tableau indiquée par un indice de début et un indice de fin) de taille n et un indice $r \in \llbracket 0; n-1 \rrbracket$, et calcule le r -ème plus petit élément de ce tableau. Pour $r = \lfloor \frac{n}{2} \rfloor$, il s'agit d'une médiane, mais r peut prendre n'importe quelle valeur et ça sera le cas lors des appels récursifs.

Le cas de base est $n \leq 5$. Alors, on trie simplement le tableau (peu importe l'algorithme) et on renvoie l'élément de rang r

Voici comment on procède dans les autres cas :

- a) On divise le tableau en $\lceil \frac{n}{5} \rceil$ sous-tableaux de longueur 5 (ou moins pour le dernier sous-tableau).
- b) On trie chaque sous-tableau (peu importe l'algorithme) et on calcule sa médiane. On construit A' le tableau des $\lceil \frac{n}{5} \rceil$ médianes calculées.
- c) On calcule la médiane de A' avec un appel récursif. Notons-la x .
- d) On applique l'algorithme du drapeau néerlandais à A et x , et on calcule i et j tels que tous les éléments de A de rang entre i inclus et j exclus sont égaux à x .
- e) On a alors trois cas possibles :
 - Si $r \in \llbracket i; j-1 \rrbracket$ on renvoie x .
 - Si $r < i$, on appelle récursivement la fonction pour chercher l'élément de rang r dans le sous-tableau de A qui contient i éléments, tous strictement inférieurs à x . On renvoie cet élément.
 - Si $r \geq j$, on appelle récursivement la fonction pour chercher l'élément de rang $r - j$ le sous-tableau de A qui contient $n - j$ éléments, tous strictement supérieurs à x . On renvoie cet élément.
6. Justifier que quelque soit l'algorithme de tri utilisé dans le cas de base ainsi qu'à l'étape b) du cas général, l'ordre de grandeur de la complexité ne va pas changer.
7. Une particularité de cet algorithme par rapport à un algorithme Diviser pour Régner ordinaire est que les étapes sont mélangées plutôt que d'être clairement séparées en deux ou trois groupes successifs. Indiquer en justifiant pour chacune des étapes s'il s'agit de Diviser, Régner, ou Rassembler.
8. Calculer la complexité totale des étapes qui ne sont pas Régner, et montrer qu'elle est linéaire en la taille du tableau.
9. a) Parmi les éléments de A' , combien sont inférieurs ou égaux à x ?
 En déduire que au moins $\left\lceil \frac{3n}{10} \right\rceil - 2$ éléments de A sont inférieurs ou égaux à x , donc que $j \geq \left\lceil \frac{3n}{10} \right\rceil - 2$.
 b) Montrer de même que $i \leq \left\lfloor \frac{7n}{10} \right\rfloor + 2$.
10. On note désormais $C'(n)$ la complexité dans le pire cas de l'algorithme pour une portion de tableau de taille au plus n . Par définition, C' est croissante sur $\mathbb{N}$.
 Montrer que $C'(n) = C\left(\left\lceil \frac{n}{5} \right\rceil\right) + C\left(\left\lfloor \frac{7n}{10} \right\rfloor + 2\right) + \mathcal{O}(n)$.
11. Montrer que $C'(n) = \mathcal{O}(n)$ (on procèdera comme usuellement par récurrence en fixant une constante bien choisie. On n'hésitera pas à choisir la constante de façon à initialiser la récurrence sur plusieurs valeurs de n). En déduire en une ligne que $C(n) = \Theta(n)$.

4 Implémentation

Pour toutes les questions qui suivent, référez-vous à la spécification des fonctions dans `tri_rapide.mli`.

12. Écrire une fonction `ieme_elt_petit_tableau` qui prend en argument un tableau `a`, deux indices `deb` et `fin` tels que $\text{deb} < \text{fin} \leq \text{deb} + 5$, et un entier $r \in \llbracket 0; \text{fin} - \text{deb} - 1 \rrbracket$. Cette fonction trie la portion de `a` entre les indices `deb` inclus et `fin` exclus, puis renvoie l'élément d'indice r dans cette portion.
13. Écrire une fonction `tab_medians` qui prend en argument `a`, `deb` et `fin`, et construit le tableau médianes des sous-tableaux de `a` de taille 5 entre `deb` et `fin`, comme dans l'étape b) de l'algorithme "médiane des médianes".
14. Implémenter l'algorithme comme la fonction récursive `ieme_elt`, qui prend en argument `a`, `deb` et `fin`, et un entier $r \in \llbracket 0; \text{fin} - \text{deb} - 1 \rrbracket$. Cette fonction échange des éléments de `a` entre les indices `deb` et `fin` (fin exclus) de façon non spécifiée, puis renvoie la valeur de rang r dans cette portion.
15. Écrire une fonction `tri_rapide_deterministe`, similaire à la fonction fournie mais qui utilise `ieme_elt` pour choisir le pivot.

5 Une conclusion décevante

Cet algorithme est (à mon humble avis) une très belle construction théorique, mais il est peu utilisé en pratique. Une des raisons est que c'est simplement plus lent que d'utiliser la version randomisée du tri rapide, du moins avec très très grande probabilité.

16. Une autre raison de ne pas utiliser cet algorithme pour des applications concrètes est qu'il a perdu le principal avantage qu'a le tri rapide sur le tri fusion, à savoir son utilisation de la mémoire. Pouvez-vous expliquer pourquoi ?